

Week_6_GIT/Exercise_4_Merge_Conflict_Resolution

Step 1: Open Git Bash

Launch **Git Bash**.

Step 2: Navigate to Your Repository

```
cd ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkilling
```

Verify that the repository is clean.

```
git status
```

Expected Output

On branch main

nothing to commit, working tree clean

CTS first requires verifying that the master (main) branch is in a clean state.

Part 1 – Create a Branch

Step 3: Create a New Branch

Create a branch named **GitWork**.

```
git branch GitWork
```

Step 4: Switch to the Branch

```
git checkout GitWork
```

or

```
git switch GitWork
```

Expected Output

Switched to branch 'GitWork'

Step 5: Create hello.xml

```
echo "<message>Hello from GitWork</message>" > hello.xml
```

Verify the file.

```
cat hello.xml
```

Expected Output

```
<message>Hello from GitWork</message>
```

CTS specifies creating a file named **hello.xml** in the branch.

```
HP@SaiKiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillig (main|SPARSE)
$ git status
On branch main
You are in a sparse checkout with 100% of tracked files present.

nothing to commit, working tree clean

HP@SaiKiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillig (main|SPARSE)
$ cd Week_6_GIT/Exercise_4_Merge_Conflict_Resolution/

HP@SaiKiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillig/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (main|SPARSE)
$ git branch GitWork

HP@SaiKiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillig/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (main|SPARSE)
$ git switch GitWork
Switched to branch 'GitWork'

HP@SaiKiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillig/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (GitWork|SPARSE)
$ echo "<message>Hello from GitWork</message>" > hello.xml

HP@SaiKiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillig/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (GitWork|SPARSE)
$ cat hello.xml
<message>Hello from GitWork</message>

HP@SaiKiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillig/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (GitWork|SPARSE)
$ |
```

Step 6: Modify hello.xml

Replace the contents.

```
echo "<message>Updated from GitWork Branch</message>" > hello.xml
```

Check repository status.

```
git status
```

Expected Output

```
modified: hello.xml
```

CTS requires updating the file and observing the status.

```

HP@Saikiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillling/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (Gitwork|SPARSE)
$ git status -u
On branch GitWork
You are in a sparse checkout with 100% of tracked files present.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    Created_Gitwork_Branch.png
    hello.xml

nothing added to commit but untracked files present (use "git add" to track)

HP@Saikiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillling/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (Gitwork|SPARSE)
$ echo "<message>Updated from GitWork Branch</message>" > hello.xml

HP@Saikiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillling/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (Gitwork|SPARSE)
$ git status -u
On branch GitWork
You are in a sparse checkout with 100% of tracked files present.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    Created_Gitwork_Branch.png
    hello.xml

nothing added to commit but untracked files present (use "git add" to track)

HP@Saikiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillling/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (Gitwork|SPARSE)
$ git status
On branch GitWork
You are in a sparse checkout with 100% of tracked files present.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    ./

nothing added to commit but untracked files present (use "git add" to track)

```

Step 7: Commit the Branch Changes

git add hello.xml

git commit -m "Update hello.xml in GitWork branch"

Expected Output

[GitWork xxxxxx]

Update hello.xml in GitWork branch

Part 2 – Modify Main Branch

Step 8: Switch Back to Main

git checkout main

or

git switch main

Step 9: Create the Same File with Different Content

echo "<message>Hello from Main Branch</message>" > hello.xml

Now overwrite it with different content.

echo "<message>Updated from Main Branch</message>" > hello.xml

Verify:

```
cat hello.xml
```

Expected Output

```
<message>Updated from Main Branch</message>
```

The CTS exercise instructs adding the same file with different content in the master branch to intentionally create a conflict.

Step 10: Commit the Main Branch Changes

```
git add hello.xml
```

```
git commit -m "Update hello.xml in main branch"
```

Expected Output

```
[main xxxxxx]
```

Update hello.xml in main branch

```
HP@SaiKiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillig/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (GitWork|SPARSE)
$ git add hello.xml
warning: in the working copy of 'Week_6_GIT/Exercise_4_Merge_Conflict_Resolution/hello.xml', LF will be replaced by CRLF the next time Git touches it

HP@SaiKiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillig/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (GitWork|SPARSE)
$ git commit -m "Update hello.xml in GitWork branch"
[GitWork 9937f29] Update hello.xml in GitWork branch
1 file changed, 1 insertion(+)
create mode 100644 Week_6_GIT/Exercise_4_Merge_Conflict_Resolution/hello.xml

HP@SaiKiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillig/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (GitWork|SPARSE)
$ git switch main
Switched to branch 'main'

HP@SaiKiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillig/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (main|SPARSE)
$ echo "<message>Hello from Main Branch</message>" > hello.xml

HP@SaiKiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillig/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (main|SPARSE)
$ echo "<message>Updated from Main Branch</message>" > hello.xml

HP@SaiKiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillig/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (main|SPARSE)
$ cat hello.xml
<message>Updated from Main Branch</message>

HP@SaiKiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillig/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (main|SPARSE)
$ git add hello.xml
warning: in the working copy of 'Week_6_GIT/Exercise_4_Merge_Conflict_Resolution/hello.xml', LF will be replaced by CRLF the next time Git touches it

HP@SaiKiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillig/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (main|SPARSE)
$ git commit -m "Update hello.xml in main branch"
[main c5a5b58] Update hello.xml in main branch
1 file changed, 1 insertion(+)
create mode 100644 Week_6_GIT/Exercise_4_Merge_Conflict_Resolution/hello.xml

HP@SaiKiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillig/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (main|SPARSE)
$ |
```

Part 3 – View Branch History

Step 11: Display the Commit Graph

```
git log --oneline --graph --decorate --all
```

Example Output

```
* abc1234 (HEAD -> main)
```

* def5678 (GitWork)

* ghi9012 Initial Commit

CTS instructs viewing the complete commit graph before merging.

```
MINGW64:/c:/Users/HP/OneDrive/Desktop/CTS_DN-5.0_DeepSkillig/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution
* c5a5b58 (HEAD -> main) Update hello.xml in main branch
| * 9937f29 (GitWork) Update hello.xml in GitWork branch
|/
* 40f0880 (origin/main, origin/HEAD) Completed Exercise 3 Branching and Merging
* 57916d2 Add branch.txt in GitNewBranch in Ex3
* 591e210 Removed branch.txt from Root folder
* 41d392f Add branch.txt in GitNewBranch
* 4ca3e3c Remove .gitignore
* 81256b4 Create README.md for Git Ignore exercise
* c355346 Added PDF and comparison Screenshot
* abc4db5 Add screenshots of Exercise2
* 80bd781 Add .gitignore to ignore log files and folders
* d4eadd9 Added pdf for Browser compatible view without downloading the docx
* 735776b Added README.md for Week_6_GIT/Exercise_1
* 2be75d4 Ignore Microsoft Office temporary files
* f5c3215 Delete Week_6_GIT/Exercise_1/~$ercise_1.docx
* 21071a8 Completed Exercisel added document and screenshots
* 1931248 Completed Exercisel
* 051e890 Added README.md for Week_5_ReactJS_HOL/Exercise_17_Fetch_User_Data_from_REST_API
* 63b262d Added Week 5 Exercise 17 Fetch User Data from REAST API
* 23a3b8d Added README.md for Week_5_ReactJS_HOL/Exercise_16_Create_mailregisterapp/
* 25c7e3d Added Week 5 Exercise 16 Create Mail Register App
* aad9123 Added README.md for Week_5_ReactJS_HOL/Exercise_15_Create_ticketraisingapp
* 4bb799d Added Week 5 Exercise 15 Create ticket raising app
* 2a0614b Added README.md for Week_5_ReactJS_HOL/Exercise_14_React_Context_API
* 70cbe8a Added Week 5 Exercise 14 React Context API
* 23f33c3 Added README.md for Week_5_ReactJS_HOL/Exercise_13_React_Conditional_Rendering_Lists_and_Keys
* 544a05f Added Week 5 Exercise 13 React Coditional Rendering Lists and Keys
* 6403ae4 Added README.md for Week_5_ReactJS_HOL/Exercise_12_React_Conditional_Rendering/
* 6e73759 Added Week 5 Exercise 12 react Conditional rendering
* 2825923 Added README.md for Week_5_ReactJS_HOL/Exercise_11_React_Events
```

Part 4 – Compare the Branches

Step 12: Compare Using Git Diff

```
git diff main GitWork
```

Git displays the differences between the two branches.

CTS requires checking the differences using the Git diff tool.

Step 13: Compare Using P4Merge (Optional)

If P4Merge is configured:

```
git difftool main GitWork
```

This opens a graphical comparison of the conflicting files.

CTS recommends P4Merge for better visualization.

Part 5 – Create the Merge Conflict

Step 14: Merge GitWork into Main

```
git merge GitWork
```


Expected Output

Auto-merging hello.xml

CONFLICT (content): Merge conflict in hello.xml

Automatic merge failed; fix conflicts and then commit the result.

This conflict occurs because both branches modified the same file differently, which is the intended outcome of this exercise.

Step 15: Observe the Conflict Markers

Open the file:

```
cat hello.xml
```

Git inserts conflict markers similar to:

```
<<<<<<< HEAD
```

```
<message>Updated from Main Branch</message>
```

```
=====
```

```
<message>Updated from GitWork Branch</message>
```

```
>>>>>>> GitWork
```

These markers indicate the conflicting changes.

CTS refers to observing the Git markup after merging.

```
HP@SaiKiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillling/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (main|SPARSE)
$ git diff main GitWork
diff --git a/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution/hello.xml b/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution/hello.xml
index d9476a1..064b23b 100644
--- a/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution/hello.xml
+++ b/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution/hello.xml
@@ -1,1 @@
- <message>Updated from Main Branch</message>
+ <message>Hello from Main Branch</message>

HP@SaiKiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillling/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (main|SPARSE)
$ git merge GitWork
Auto-merging Week_6_GIT/Exercise_4_Merge_Conflict_Resolution/hello.xml
CONFLICT (add/add): Merge conflict in Week_6_GIT/Exercise_4_Merge_Conflict_Resolution/hello.xml
Automatic merge failed; fix conflicts and then commit the result.

HP@SaiKiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillling/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (main|SPARSE|MERCING)
$ cat hello.xml
<<<<<<< HEAD
<message>Updated from Main Branch</message>
=====
<message>Hello from Main Branch</message>
>>>>>>> GitWork

HP@SaiKiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillling/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (main|SPARSE|MERCING)
$ |
```

Part 6 – Resolve the Conflict

Step 16: Edit hello.xml

Remove the conflict markers and keep the desired content.

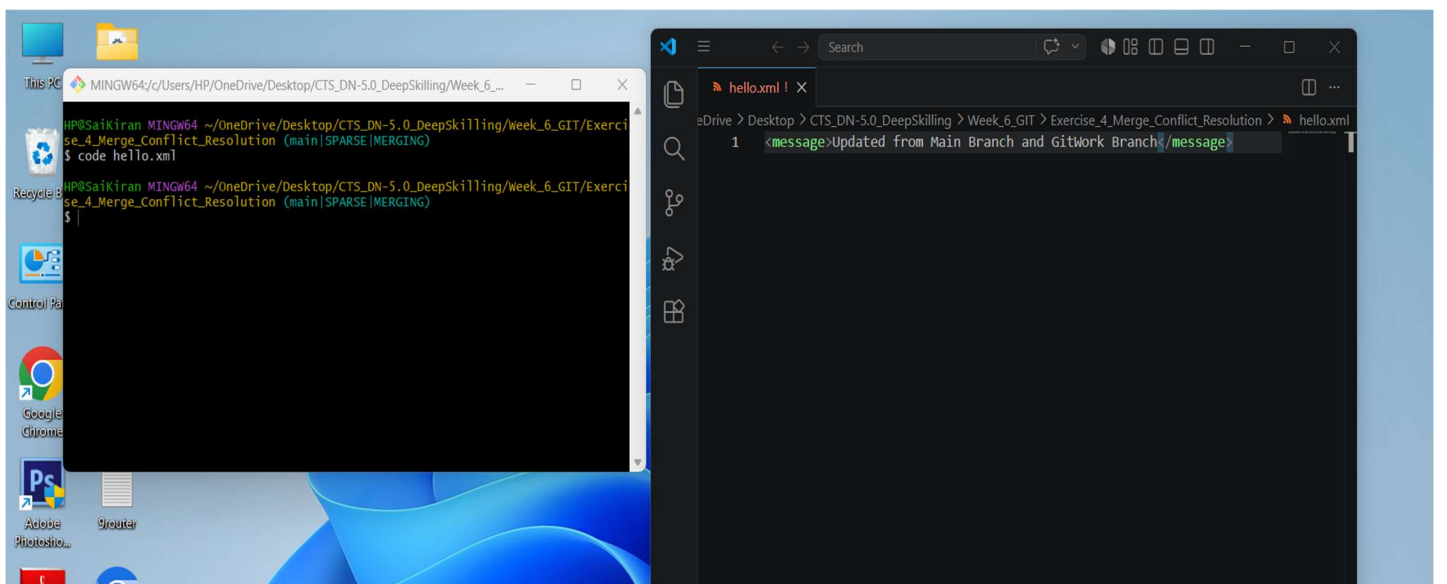
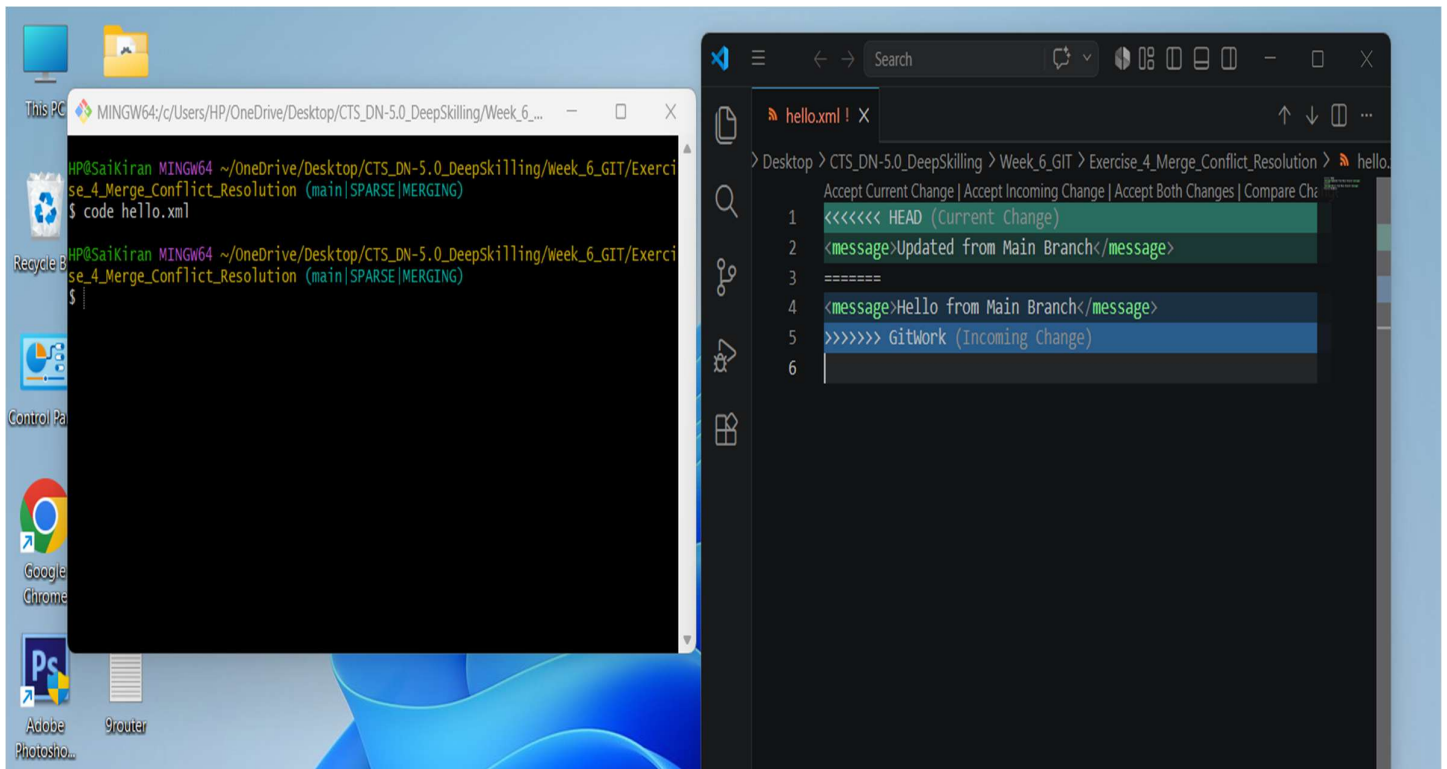
Example:

```
<message>Updated from Main Branch and GitWork Branch</message>
```

Save the file.

If using **P4Merge**, resolve the conflict using its 3-way merge interface.

CTS specifically requires using a 3-way merge tool to resolve the conflict.



Step 17: Stage the Resolved File

```
git add hello.xml
```

Step 18: Complete the Merge

```
git commit -m "Resolve merge conflict in hello.xml"
```

Expected Output

```
[main xxxxxx]
```

Resolve merge conflict in hello.xml

CTS instructs committing the resolved merge.

Part 7 – Ignore Backup Files

Step 19: Check Repository Status

```
git status
```

If your merge tool created backup files (for example, hello.xml.orig), add them to .gitignore.

Open or create .gitignore and add:

```
*.orig
```

Stage the .gitignore file:

```
git add .gitignore
```

Commit:

```
git commit -m "Ignore backup files"
```

CTS requires adding backup files to .gitignore and committing the changes.

Part 8 – Clean Up

Step 20: List All Branches

```
git branch
```

Example Output

```
GitWork
```

```
* main
```



```
HP@SaiKiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillling/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (main|SPARSE|MERGING)
$ git add hello.xml

HP@SaiKiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillling/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (main|SPARSE|MERGING)
$ git commit -m "Resolve merge conflict in hello.xml"
[main 48d5447] Resolve merge conflict in hello.xml

HP@SaiKiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillling/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (main|SPARSE)
$ git status
On branch main
You are in a sparse checkout with 100% of tracked files present.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    1_Created_GitWork_Branch.png
    2_Updated_hello.xml.png
    3_Committed_hello.xml_in_main_branch.png
    4_git_log.png
    5_Merge_conflict.png
    6_Editing_in_VSC.png
    8_Editted_hello.xml.png

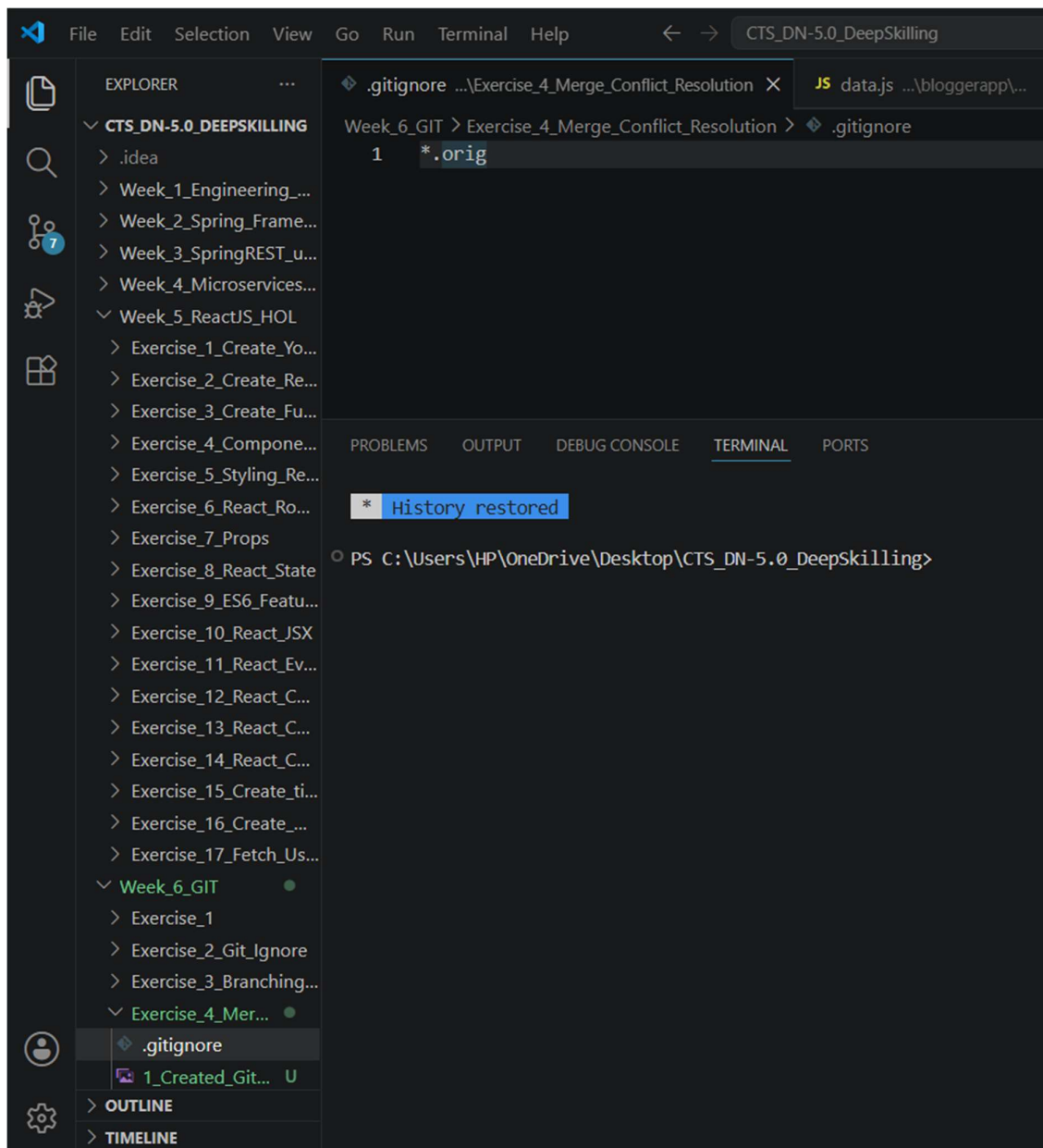
nothing added to commit but untracked files present (use "git add" to track)

HP@SaiKiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillling/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (main|SPARSE)
$ git add .gitignore

HP@SaiKiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillling/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (main|SPARSE)
$ git commit -m "Ignore backup files"
[main 39f7f9c] Ignore backup files
1 file changed, 1 insertion(+)
create mode 100644 Week_6_GIT/Exercise_4_Merge_Conflict_Resolution/.gitignore

HP@SaiKiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillling/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (main|SPARSE)
$ git branch
  GitWork
* main

HP@SaiKiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillling/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (main|SPARSE)
$ |
```



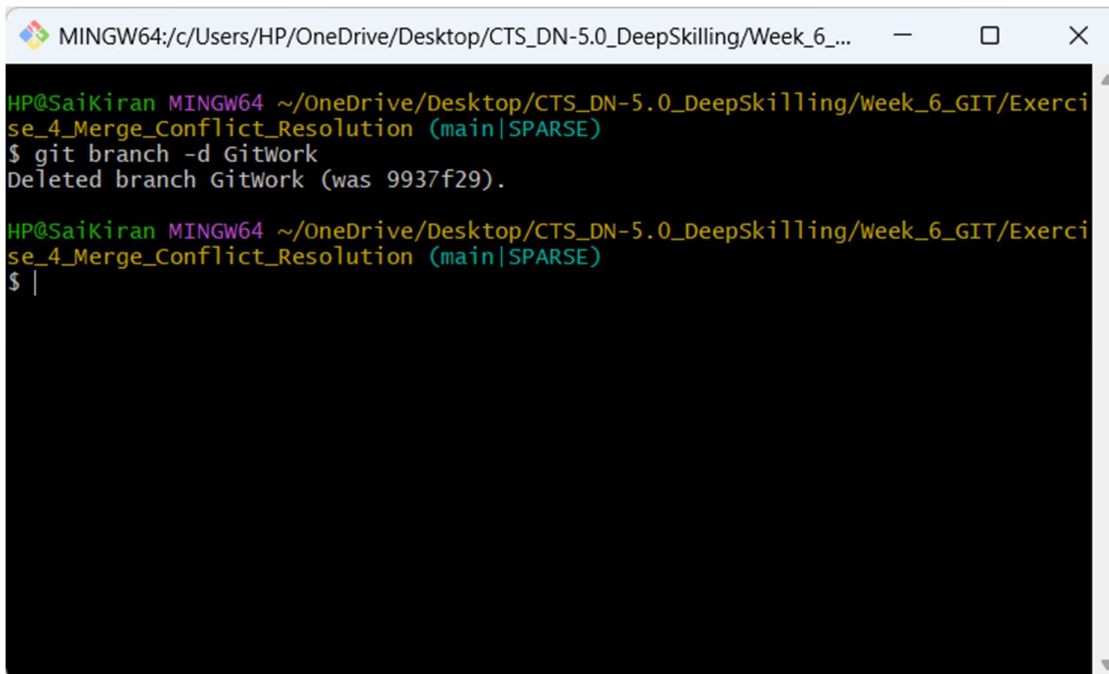
Step 21: Delete the Merged Branch

git branch -d GitWork

Expected Output

Deleted branch GitWork (was abc1234).

CTS instructs deleting the merged branch.

A screenshot of a Windows command prompt window. The title bar shows the path "MINGW64:/c/Users/HP/OneDrive/Desktop/CTS_DN-5.0_DeepSkillling/Week_6_...". The prompt is "HP@Saikiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillling/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (main|SPARSE)". The user enters the command "\$ git branch -d GitWork". The output is "Deleted branch GitWork (was 9937f29).". The prompt returns to the same state.

```
HP@Saikiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillling/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (main|SPARSE)
$ git branch -d GitWork
Deleted branch GitWork (was 9937f29).

HP@Saikiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillling/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (main|SPARSE)
$ |
```

Step 22: View the Final Commit Graph

`git log --oneline --graph --decorate`

Example Output

- * 9ab4567 (HEAD -> main) Resolve merge conflict in hello.xml
- * 8cd3456 Merge branch 'GitWork'
- * 7ef2345 Update hello.xml in main branch
- * 6de1234 Update hello.xml in GitWork branch
- * 5bc0123 Initial Commit

This confirms the merge conflict was resolved successfully and the history reflects the merge.

```
MINGW64:/c:/Users/HP/OneDrive/Desktop/CTS_DN-5.0_DeepSkillig/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution
```

```
* 39f7f9c (HEAD -> main) Ignore backup files
* 48d5447 Resolve merge conflict in hello.xml
| \
| * 9937f29 Update hello.xml in GitWork branch
* | c5a5b58 Update hello.xml in main branch
| /
* 40f0880 (origin/main, origin/HEAD) Completed Exercise 3 Branching and Merging
* 57916d2 Add branch.txt in GitNewBranch in Ex3
* 591e210 Removed branch.txt from Root folder
* 41d392f Add branch.txt in GitNewBranch
* 4ca3e3c Remove .gitignore
* 81256b4 Create README.md for Git Ignore exercise
* c355346 Added PDF and comparison Screenshot
* abc4db5 Add screenshots of Exercise2
* 80bd781 Add .gitignore to ignore log files and folders
* d4eadd9 Added pdf for Browser compatible view without downloading the docx
* 735776b Added README.md for Week_6_GIT/Exercise_1
* 2be75d4 Ignore Microsoft Office temporary files
* f5c3215 Delete Week_6_GIT/Exercise_1/~$ercise_1.docx
* 21071a8 Completed Exercise1 added document and screenshots
* 1931248 Completed Exercise1
* 051e890 Added README.md for Week_5_ReactJS_HOL/Exercise_17_Fetch_User_Data_from_REST_API
...skipping...
* 39f7f9c (HEAD -> main) Ignore backup files
* 48d5447 Resolve merge conflict in hello.xml
| \
| * 9937f29 Update hello.xml in GitWork branch
* | c5a5b58 Update hello.xml in main branch
| /
* 40f0880 (origin/main, origin/HEAD) Completed Exercise 3 Branching and Merging
* 57916d2 Add branch.txt in GitNewBranch in Ex3
* 591e210 Removed branch.txt from Root folder
* 41d392f Add branch.txt in GitNewBranch
* 4ca3e3c Remove .gitignore
* 81256b4 Create README.md for Git Ignore exercise
* c355346 Added PDF and comparison Screenshot
* abc4db5 Add screenshots of Exercise2
* 80bd781 Add .gitignore to ignore log files and folders
* d4eadd9 Added pdf for Browser compatible view without downloading the docx
```

Push the Changes to GitHub

git pull origin main

git push origin main

```
MINGW64:/c:/Users/HP/OneDrive/Desktop/CTS_DN-5.0_DeepSkillig/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution
```

```
HP@SaiKiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillig/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (main|SPARSE)
$ git add .

HP@SaiKiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillig/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (main|SPARSE)
$ git commit -m "Adding Screenshots to the Exercise 4"
[main 136ccf5] Adding Screenshots to the Exercise 4
10 files changed, 0 insertions(+), 0 deletions(-)
create mode 100644 Week_6_GIT/Exercise_4_Merge_Conflict_Resolution/10_deleted_GitWork_branch.png
create mode 100644 Week_6_GIT/Exercise_4_Merge_Conflict_Resolution/11_Git_log.png
create mode 100644 Week_6_GIT/Exercise_4_Merge_Conflict_Resolution/1_Created_GitWork_Branch.png
create mode 100644 Week_6_GIT/Exercise_4_Merge_Conflict_Resolution/2_Updated_hello.xml.png
create mode 100644 Week_6_GIT/Exercise_4_Merge_Conflict_Resolution/3_Committed_hello.xml_in_main_branch.png
create mode 100644 Week_6_GIT/Exercise_4_Merge_Conflict_Resolution/4_git_log.png
create mode 100644 Week_6_GIT/Exercise_4_Merge_Conflict_Resolution/5_Merge_conflict.png
create mode 100644 Week_6_GIT/Exercise_4_Merge_Conflict_Resolution/6_Editing_in_VSC.png
create mode 100644 Week_6_GIT/Exercise_4_Merge_Conflict_Resolution/8_Editted_hello.xml.png
create mode 100644 Week_6_GIT/Exercise_4_Merge_Conflict_Resolution/9_Resolved_Conflict.png

HP@SaiKiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillig/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (main|SPARSE)
$ git pull origin main
remote: Enumerating objects: 8, done.
remote: Counting objects: 100% (8/8), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 5 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (5/5), 8.00 KiB | 431.00 KiB/s, done.
From https://github.com/G-Manikanta1729/CTS_DN-5.0_DeepSkillig
* branch main -> FETCH_HEAD
* 40f0880..ae2185c main -> origin/main
Merge made by the 'ort' strategy.
.../Exercise_3_Branching_and_Merging/README.md | 1492 +++++
1 file changed, 1492 insertions(+)
create mode 100644 Week_6_GIT/Exercise_3_Branching_and_Merging/README.md
git pgit pgit p
HP@SaiKiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillig/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (main|SPARSE)
$ git pull origin main --rebase
From https://github.com/G-Manikanta1729/CTS_DN-5.0_DeepSkillig
* branch main -> FETCH_HEAD
Deletion of directory 'Week_6_GIT/Exercise_4_Merge_Conflict_Resolution' failed. Should I try again? (y/n) n
Auto-merging Week_6_GIT/Exercise_4_Merge_Conflict_Resolution/hello.xml
CONFLICT (add/add): Merge conflict in Week_6_GIT/Exercise_4_Merge_Conflict_Resolution/hello.xml
error: could not apply 9937f29... Update hello.xml in GitWork branch
hint: Resolve all conflicts manually, mark them as resolved with
hint: "git add/rm <conflicted_files>", then run "git rebase --continue".
hint: You can instead skip this commit: run "git rebase --skip".
hint: To abort and get back to the state before "git rebase", run "git rebase --abort".
hint: Disable this message with "git config set advice.mergeConflict false"
Could not apply 9937f29... # Update hello.xml in GitWork branch

HP@SaiKiran MINGW64 ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillig/Week_6_GIT/Exercise_4_Merge_Conflict_Resolution (main|SPARSE|REBASE 2/4)
$ git push origin main
Enumerating objects: 41, done.
Counting objects: 100% (41/41), done.
Delta compression using up to 12 threads
Compressing objects: 100% (31/31), done.
Writing objects: 100% (37/37), 980.13 KiB | 31.62 MiB/s, done.
Total 37 (delta 13), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (13/13), completed with 2 local objects.
```

Final Project Structure

CTS_DN-5.0_DeepSkillig/

|

|— Week_1_Engineering_Concepts/

|— Week_2_Spring_Framework/

|— Week_3_Spring_REST/

|— Week_4_Microservices/

|— Week_5_ReactJS_HOL/

|— Week_6_GIT/

 |— Exercise_4_Merge_Conflict_Resolution/

 |— hello.xml

 |— .gitignore

Command Summary

cd ~/OneDrive/Desktop/CTS_DN-5.0_DeepSkillig

git status

git branch GitWork

git checkout GitWork

echo "<message>Hello from GitWork</message>" > hello.xml

echo "<message>Updated from GitWork Branch</message>" > hello.xml

git add hello.xml

git commit -m "Update hello.xml in GitWork branch"

git checkout main

```
echo "<message>Updated from Main Branch</message>" > hello.xml
```

```
git add hello.xml
```

```
git commit -m "Update hello.xml in main branch"
```

```
git log --oneline --graph --decorate --all
```

```
git diff main GitWork
```

```
git difftool main GitWork    # Optional
```

```
git merge GitWork
```

```
# Resolve conflict in hello.xml
```

```
git add hello.xml
```

```
git commit -m "Resolve merge conflict in hello.xml"
```

```
git status
```

```
git add .gitignore
```

```
git commit -m "Ignore backup files"
```

```
git branch
```

```
git branch -d GitWork
```

```
git log --oneline --graph --decorate
```



```
git pull origin main
```

```
git push origin main
```

This implementation follows the CTS Git Hands-on sequence: verify a clean repository, create a branch, make conflicting changes in both branches, observe the conflict, resolve it using a merge tool or manually, ignore backup files, delete the merged branch, and inspect the final commit history.